

ARQUITETURA DE SOFTWARE

DEFININDO: ARQUITETURA DE SOFTWARE

As decisões da arquitetura formam os limites do sistema e orientam as equipes de desenvolvimento sobre o que é ou não permitido.

DEFININDO: ARQUITETURA DE SOFTWARE

Gerenciamento de arquiteturas

Arquiteturas corporativas

Arquitetura de informação

Arquitetura de infraestrutura

Arquitetura de aplicação

Tendências da arquitetura

Arquitetura orientada a serviços

Virtualização

Computação em grade

- **DEFININDO: ARQUITETURA DE SOFTWARE**

Arquiteturas corporativas – incluem os planos de como uma organização deve construir, implantar, utilizar e compartilhar seus dados, processos e ativos de TI.

Principais metas da arquitetura corporativa:

- reduzir custos
- melhorar a produtividade
- criar vantagens competitivas
- gerar fluxos de receitas
- alinhar os processos

DEFININDO: ARQUITETURA DE SOFTWARE

Arquitetura de informação – identifica onde e como as informações importantes, como registros de clientes, serão mantidas e protegidas.

Arquitetura de informação deve focar:

- Backup e recuperação
- Recuperação de desastres
- Segurança da informação

DEFININDO: ARQUITETURA DE SOFTWARE

Backup – cópia exata de um sistema de informação.

Recuperação – possibilidade de obter um sistema restabelecido e em funcionamento em caso de queda ou falha e inclui a restauração do backup de informações.

Duas técnicas utilizadas para ajudar em caso de falha do sistema são a tolerância a falhas e o failover (...que é a capacidade de transferir automaticamente o tráfego de um link para outro em caso de uma falha ou indisponibilidade de um dos links.)

DEFININDO: ARQUITETURA DE SOFTWARE

Segurança da Informação - Boas arquiteturas de informação incluem:

- Um forte plano de segurança de informação
- Gerenciamento do acesso dos usuários
- Atualização de softwares antivírus e de correção

DEFININDO: ARQUITETURA DE SOFTWARE

Arquitetura de infraestrutura – inclui o equipamento de hardware, software e telecomunicações que, quando combinados, fornecem a base subjacente para sustentar as metas da empresa.

Principais características de uma arquitetura de infraestrutura sólida:
flexibilidade; escalabilidade; confiabilidade; disponibilidade;
desempenho.

DEFININDO: ARQUITETURA DE SOFTWARE

Arquitetura de aplicação – determina como os aplicativos se integram e relacionam entre si.

Serviços web – contêm um repertório de dados e recursos processuais que utiliza protocolos compartilhados e padrões que permitem que diferentes aplicações partilhem dados e serviços.

Sistema aberto – descreve hardwares e softwares de TI não proprietários, disponibilizando os padrões e rotinas a partir dos quais eles operam, tornando mais fácil integrá-los.

DEFININDO: ARQUITETURA DE SOFTWARE

Arquiteturas orientadas a serviços (SOA) uma abordagem arquitetural da TI orientada aos negócios que sustenta a integração de um negócio como serviços (ou tarefas) conectadas ou iterativas.

SOA garante que os sistemas podem se adaptar veloz, fácil e economicamente para suportar as rápidas necessidades dos negócios. Auxilia ainda em aumentar a flexibilidade de seus processos.

DEFININDO: ARQUITETURA DE SOFTWARE

Os principais conceitos técnicos da SOA são:

Serviços – pode ser uma tarefa de negócio, como a avaliação de crédito de clientes potenciais no momento em que abrem uma conta.

Interoperabilidade – é a capacidade de dois ou mais sistemas de computadores compartilharem dados e recursos, mesmo que eles sejam feitos por diferentes fabricantes.

Baixo acoplamento – possibilidade de que os serviços sejam combinados para criar serviços compostos, ou desagregados, com a mesma facilidade, em seus componentes funcionais.

DEFININDO: ARQUITETURA DE SOFTWARE

Virtualização – estrutura de divisão dos recursos de um computador em vários ambientes de execução.

Sistema de virtualização – possibilidade de tratar um único computador como se fosse uma coleção de computadores separados (“máquinas virtuais”).

DEFININDO: ARQUITETURA DE SOFTWARE

Benefícios de Negócios da Virtualização

Tendências levaram a virtualização para o centro das atenções:

- Hardware em renovação
- Data centers sem espaço
- Aumento dos custos de energia
- Custos do sistema de administração
- Rápida implantação de aplicativos
- Balanceamento dinâmico de carga
- Recuperação simplificada de desastres

DEFININDO: ARQUITETURA DE SOFTWARE

Computação em grade – agregação de computação geograficamente dispersa, armazenamento e recursos de rede, coordenada para oferecer melhor desempenho, maior qualidade de serviço, melhor utilização e um acesso mais fácil aos dados.

DEFININDO: ARQUITETURA DE SOFTWARE

Benefícios de Negócios da Computação em Grade

Aumentar a produtividade e a colaboração de organizações virtuais e respectivos recursos de informática e de dados.

Permitir que amplos departamentos dispersos e empresas criem organizações virtuais para compartilhar dados e recursos.

DEFININDO: ARQUITETURA DE SOFTWARE

Benefícios de Negócios da Computação em Grade

Construir arquiteturas operacionais robustas, infinitamente flexíveis e resilientes.

Proporcionar o acesso instantâneo à computação massiva e aos recursos de dados.

Alavancar os investimentos de capital existentes.

ARQUITETURA DE SOFTWARE

ESCALABILIDADE

TÓPICOS:

Tecnologia da Informação e o seu papel na escalabilidade

Etapas da cadeia de valor e a participação da Tecnologia da Informação

Profissionais ligados à tecnologia em cada etapa

Tudo o que você precisa aprender é trabalhar com Escalabilidade:

Empresas Escaláveis

Negócios Escaláveis

Produtos Escaláveis

Times Escaláveis

Softwares Escaláveis

Tudo o que você precisa aprender é trabalhar com Escalabilidade:

Linguagens Escaláveis
Códigos Escaláveis
Infraestruturas Escaláveis
P & D
Pessoas Escaláveis

A escalabilidade é um termo que se refere à produção em escala e seu objetivo é permitir aplicar esse conceito nos negócios para que eles possam aumentar em larga escala, com custos reduzidos e melhores resultados.

Ser capaz de escalar os serviços para atender o aumento repentino de clientes passou a ser fundamental para todas as empresas que querem manter sua competitividade no mercado.

Empresas Escaláveis

A escalabilidade é a possibilidade que uma empresa tem de crescer, agregar valor ao seu negócio e atender as demandas dos clientes sem aumentar expressivamente os seus custos.

Sendo assim, quanto mais escalável for uma corporação, maiores são as chances de aumentar os lucros.

Empresas Escaláveis

Uma startup escalável é, em termos muito simples, aquela que consegue expandir (muito) o seu número de clientes, usuários e/ou faturamento de forma acelerada, sem precisar aumentar seus custos na mesma proporção.

Empresas Escaláveis

Grandes corporações precisam de Agilidade Organizacional.

Empresas Escaláveis

Metodologias de Trabalho: Objectives and Key Results

Realizar um planejamento estratégico com definições claras dos objetivos

Elaboração de hipóteses que poderão levar a empresa a alcançar as suas estratégias

Time trabalhando totalmente em busca de atender o objetivo da empresa

Empresas Escaláveis

Profissionais com Skill de TI (ou não)

Agilista / Agile Master - Aplicador de práticas ágeis

Agile Coach - Treinador de práticas ágeis

Produtos Escaláveis

MVP - O significado de MVP é Minimum Viable Product (Produto Mínimo Viável), ou seja, um conjunto mínimo de funcionalidades para validar e testar o mais rápido possível seu produto/ideia.

Produtos Escaláveis: Método Design Thinking

Construa o que realmente vai agregar valor ao cliente

Design thinking é uma metodologia usada para a criação de novos produtos, serviços, processos ou para a resolução de problemas.

Produtos Escaláveis

Profissionais com Skill de TI (ou não)

Product Owner

Analista de Produtos

Analista de Sistemas

Negócios Escaláveis

Caminho das Corporações é a Transformação Digital

Adaptação, Evolução e Transformação

Negócios Escaláveis

Hoje, o grande desafio que as organizações enfrentam é promover uma cultura que facilite o processo de transformação que estão vivenciando, amplie as competências digitais de seu capital humano e estabeleça novos métodos de trabalho que promovam a agilidade interna e a inovação.

Negócios Escaláveis

Transição do Físico para o Digital

Mercados, empresas, times e pessoas estão na transição do FÍSICO para o Físico, habilitado e estendido pelo diGITal, orquestrados no espaço sociAL, em tempo [quase] real.

Negócios Escaláveis

Durante a coletiva de imprensa para anunciar a aquisição da NOKIA pela Microsoft, o CEO da Nokia encerrou seu discurso dizendo que “não fizemos nada de errado, mas de alguma forma perdemos”. Ao dizer isso, toda a sua equipe de gestão, incluindo ele próprio, chorou tristemente.

Negócios Escaláveis

A Nokia tem sido uma empresa respeitável. Eles não fizeram nada de errado em seus negócios, no entanto, o mundo mudou muito rápido. Seus oponentes eram muito poderosos.

Negócios Escaláveis

Eles perderam o aprendizado, perderam a mudança e, assim, perderam a oportunidade de se tornarem grandes. Eles não apenas perderam a oportunidade de ganhar muito dinheiro, como também perderam a chance de sobrevivência.

A mensagem desta história é que, se você não mudar, será removido da competição.

Negócios Escaláveis

- Segurança da Informação

 - LGPD

- Data Driven: Oriente-se pelos Dados para direcionar os negócios da Empresa

Negócios Escaláveis

-Plataforma

Consiste em um modelo de gestão de negócios no qual a organização é desenvolvida com base em uma plataforma, em vez de um produto em si.

Exemplos:

Uber - 100 carros ou 1 milhão de carros

Marketplace - 1 Vendedor ou 1 milhão de vendedores

Negócios Escaláveis

- Profissionais com Skill de TI
 - Cientista de Dados
- Analista de Segurança da Informação
 - Analista de Projetos

Times Escaláveis

- Time multidisciplinar em volta de um fluxo de valor
 - Missão clara e única
- Atuar ponto a ponto Passando o bastão dentro do time
 - Nem sempre time muito pequeno, mas o suficiente
- Time não em volta de aplicações e sim de ponta a ponta de um fluxo

Softwares Escaláveis

Pense grande: As principais vantagens de ter softwares que possam ser escaláveis é a redução de custos, tanto dos equipamentos e recursos técnicos, quanto dos humanos. A rapidez para se adaptar a mudanças ou implementá-las, sem perda da qualidade da entrega, é um grande diferencial competitivo em relação aos concorrentes.

Softwares Escaláveis

Outra vantagem é evitar problemas, como instabilidades no sistema, falhas técnicas que podem corromper ou afetar banco de dados, ou até mesmo o lançamento de produtos ou recursos que sejam muito complexos para o usuário, sobrecarregando o suporte.

- Softwares Escaláveis

Docker - Containers Serviços

Kubernetes - Orquestração

APIs

Banco de Dados escaláveis

Cloud First

Mobile First

- Softwares Escaláveis

Micro Serviços Reutilizáveis

Testes automatizados - BDD / TDD / Selenium

Deploys automatizados - Jenkins

Monitoramento - Prometheus / Grafana / Kibana

Health Check

Softwares Escaláveis

Containers: O Docker é uma plataforma de containerização e um período de execução e o Kubernetes é uma plataforma para executar e gerenciar contêineres de vários períodos de execução de contêineres.

Linguagens Escaláveis

Golang

Java

Kotlin

Java Script

C#

React Js

Python

Códigos Escaláveis

Algumas características de um código escalável:

Organizado

Bem documentado

Limpo

Não é verboso

Teste Automatizados

Códigos Escaláveis

Utilizar plataformas de hospedagem de códigos

Git - Sistema de Controle de Versões

GitHub / GitLab / Bitbucket - Plataformas de gerenciamento de Códigos

Infraestruturas Escaláveis

Filas e Mensagerias: A escalabilidade em Cloud Computing é a possibilidade de uma empresa poder ampliar ou reduzir os recursos utilizados de forma inteligente.

Essa característica dá aos usuários a capacidade de modificar a quantidade de memória, espaço de armazenamento ou capacidade de processamento do servidor sem a necessidade de uma nova configuração.

Infraestruturas Escaláveis

API Gateway: API Gateway é um serviço gerenciado que permite que desenvolvedores criem, publiquem, mantenham, monitorem e protejam APIs em qualquer escala com facilidade.

APIs agem como a “porta de entrada” para aplicativos acessarem dados, lógica de negócios ou funcionalidade de seus serviços de back-end.

Infraestruturas Escaláveis

Cloud Computing: A escalabilidade em Cloud Computing é a possibilidade de uma empresa poder ampliar ou reduzir os recursos utilizados de forma inteligente.

Infraestruturas Escaláveis

Infraestrutura On Premise: Recursos computacionais escaláveis

Nginx: Servidor leve de HTTP, proxy reverso, proxy de e-mail
IMAP/POP

Infraestruturas Escaláveis

HAProxy: Balanceador de carga e servidor proxy de alta disponibilidade para aplicativos baseados em TCP/IP e HTTP que espalha solicitações por vários servidores

Memcached: Sistema distribuído de cache em memória

Infraestruturas Escaláveis

Cultura Devops

Metodologia de desenvolvimento de software responsável por unificar as equipes de desenvolvimento e operações.

Infraestruturas Escaláveis

Bancos de dados NoSQL: São criados para modelos de dados específicos e têm esquemas flexíveis para a criação de aplicativos modernos.

Os bancos de dados NoSQL são amplamente reconhecidos por sua facilidade de desenvolvimento, funcionalidade e performance em escala.

Infraestruturas Escaláveis

Bancos de dados Relacionais: Todos os dados que podem ser agrupados e exibidos em tabelas, ou seja, com linhas e colunas, são relacionais.

Infraestruturas Escaláveis

Bancos de dados NoSQL

São criados para modelos de dados específicos e têm esquemas flexíveis para a criação de aplicativos modernos. Os bancos de dados NoSQL são amplamente reconhecidos por sua facilidade de desenvolvimento, funcionalidade e performance em escala.

ARQUITETURA DE SOFTWARE

Richards, Mark; Ford, Neal. Fundamentos da arquitetura de Software: Uma abordagem de engenharia (O'Reilly Livro 1) (Portuguese Edition) (p. 35). Alta Books. Edição do Kindle.

EXPECTATIVAS DE UM ARQUITETO

Definir o papel de um arquiteto de software se mostra tão difícil quanto definir a arquitetura de software.

Pode variar desde um programador especializado até definir a direção técnica estratégica para a empresa.

Em vez de perder tempo com a busca inútil de definir a função, recomendamos focar as expectativas de um arquiteto.

EXPECTATIVAS DE UM ARQUITETO

Existem oito expectativas principais para um arquiteto de software, independentemente de qualquer função, cargo ou descrição do trabalho.

O primeiro segredo para a eficiência e o sucesso na função de arquiteto de software depende de compreender e praticar cada uma dessas expectativas.

- 1-Tomar decisões de arquitetura
- 2-Analisar continuamente a arquitetura
- 3-Manter-se atualizado com as últimas tendências
- 4-Assegurar a conformidade com as decisões
- 5-Exposição e experiência diversificadas
- 6-Ter conhecimento sobre o domínio do negócio
- 7-Ter habilidades interpessoais
- 8-Entender e lidar bem com questões políticas

1-Tomar Decisões de Arquitetura

Um arquiteto deve estabelecer as decisões da arquitetura e os princípios do design usados para orientar as decisões de tecnologia dentro da equipe, do departamento ou em toda a empresa.

Orientação é a palavra de ordem nessa primeira expectativa. Um arquiteto deve orientar, e não especificar as escolhas da tecnologia.

1-Tomar Decisões de Arquitetura

Por exemplo, um arquiteto pode tomar a decisão de usar React.js para um desenvolvimento front-end. Nesse caso, ele está tomando uma decisão técnica, não uma decisão arquitetural ou um princípio de design que ajudará a equipe de desenvolvimento a fazer escolhas.

Um arquiteto deve instruir as equipes de desenvolvimento para usarem um framework reativo para o desenvolvimento web front-end, orientando a equipe ao fazer a escolha entre Angular, Elm, React.js ou qualquer outro framework web reativo.

1-Tomar Decisões de Arquitetura

É difícil orientar as escolhas da tecnologia com decisões da arquitetura e princípios do design. O segredo para tomar decisões arquiteturais eficientes é perguntar se a decisão da arquitetura está ajudando a orientar as equipes ao fazerem a escolha técnica certa ou se a decisão faz a escolha técnica por elas.

1-Tomar Decisões de Arquitetura

Dito isso, por vezes um arquiteto talvez precise tomar decisões de tecnologia específicas para preservar uma característica da arquitetura em particular, como escalabilidade, desempenho ou disponibilidade.

Nesse caso, ainda seria considerada uma decisão de arquitetura, mesmo que especifique uma tecnologia em particular. Em geral, os arquitetos se esforçam para encontrar a linha certa, portanto é totalmente importante estudar sobre decisões da arquitetura.

2-Analisar Continuamente a Arquitetura

Um arquiteto deve analisar continuamente a arquitetura e o ambiente de tecnologia atual, para então recomendar soluções de melhorias.

Essa expectativa de um arquiteto se refere à vitalidade da arquitetura, que avalia se uma arquitetura é viável hoje tendo sido definida há três anos ou mais, dadas as mudanças no negócio e na tecnologia.

2-Analisar Continuamente a Arquitetura

Conforme nossa experiência, poucos arquitetos focam suas energias em analisar continuamente as arquiteturas existentes. Como resultado, a maioria deles enfrenta elementos de decadência estrutural, o que ocorre quando os desenvolvedores fazem mudanças no código ou no design que impactam as características arquiteturais requeridas, como desempenho, disponibilidade e escalabilidade.

2-Analisar Continuamente a Arquitetura

Outros aspectos esquecidos dessa expectativa que os arquitetos costumam negligenciar são os ambientes de teste e lançamento. A agilidade na modificação do código tem benefícios óbvios, mas se as equipes precisam de semanas para testar as mudanças e meses para lançar, então os arquitetos não conseguem ter agilidade na arquitetura em geral.

2-Analisar Continuamente a Arquitetura

Um arquiteto deve analisar integralmente as mudanças na tecnologia e os domínios do problema para determinar a robustez da arquitetura.

Embora essa consideração apareça raramente em um anúncio de empregos, os arquitetos devem atender a essa expectativa para manter as aplicações relevantes.

3-Manter-se Atualizado com as Últimas Tendências

Um arquiteto deve ficar atualizado com as últimas tendências da tecnologia e do setor. Os desenvolvedores devem ficar atualizados com as últimas tecnologias usadas diariamente para manterem a si mesmos (e seu trabalho!) relevantes.

Um arquiteto tem um requisito ainda mais crítico de se manter atual com as últimas tendências técnicas e do setor. As decisões que um arquiteto toma tendem a ser de longo prazo e difíceis de mudar.

3-Manter-se Atualizado com as Últimas Tendências

Entender e seguir as principais tendências contribui para que ele se prepare para o futuro e tome a decisão certa.

Seguir as tendências e se manter atualizado com elas é difícil, sobretudo para um arquiteto de software.

É preciso conhecer as várias técnicas e recursos sobre como fazer isso.

4-Assegurar a Conformidade com as Decisões

Um arquiteto deve assegurar a conformidade com as decisões de arquitetura e os princípios de design.

Assegurar a conformidade significa que o arquiteto verifica continuamente se as equipes de desenvolvimento seguem as decisões da arquitetura e os princípios do design definidos, documentados e comunicados por ele.

4-Assegurar a Conformidade com as Decisões

Considere o cenário em que um arquiteto toma a decisão de restringir o acesso ao banco de dados em uma arquitetura de camadas apenas para as camadas de negócio e serviços (e não para camada de apresentação).

Isso significa que a camada de apresentação deverá passar por todas as camadas da arquitetura para fazer até o mais simples chamado do banco de dados.

Um desenvolvedor da interface do usuário poderá discordar dessa decisão e acessar o banco de dados (ou a camada de persistência) diretamente por motivos de desempenho.

4-Assegurar a Conformidade com as Decisões

Contudo, o arquiteto tomou essa decisão de arquitetura por um motivo específico: controlar as mudanças.

Fechando as camadas, as mudanças no banco de dados podem ser feitas sem impactar a camada de apresentação.

Por não assegurar a conformidade com as decisões da arquitetura, esse tipo de violação pode acontecer, a arquitetura não atenderá às características requeridas (atributos) e a aplicação ou o sistema não funcionará como esperado.

5-Exposição e Experiência Diversificadas

Um arquiteto deve se expor a tecnologias, estruturas, plataformas e ambientes múltiplos e diversificados. Essa expectativa não significa que um arquiteto deve ser especialista em toda estrutura, plataforma e linguagem, mas deve estar familiarizado pelo menos com várias tecnologias.

A maioria dos ambientes atuais é heterogênea e, no mínimo, um arquiteto deve saber como se comunicar com vários sistemas e serviços, não importando a linguagem, a plataforma e a tecnologia nas quais esses sistemas ou serviços estão escritos.

5-Exposição e Experiência Diversificadas

Um dos melhores modos de dominar essa expectativa é o arquiteto aumentar sua zona de conforto. Focar apenas uma tecnologia ou plataforma é um porto seguro.

Um arquiteto de software eficiente deve ser agressivo ao buscar oportunidades para ter experiência em várias linguagens, plataformas e tecnologias.

5-Exposição e Experiência Diversificadas

Uma boa maneira de dominar a expectativa é se concentrar na amplitude técnica, não na profundidade técnica.

Amplitude técnica inclui o que você sabe, mas não com detalhes, combinado com o que você sabe muito.

Por exemplo, é muito melhor que o arquiteto esteja familiarizado com dez produtos ‘diferentes’ e com os prós e contras associados a cada um do que ser especialista em apenas um deles.

6-Ter Conhecimento sobre o Domínio do Negócio

Um arquiteto deve ter certo nível de especialização no domínio do negócio.

Os arquitetos de software eficientes entendem não apenas de tecnologia, mas do domínio do negócio no espaço do problema.

Sem conhecimento do domínio do negócio, fica difícil entender o problema comercial, as metas e os requisitos, complicando planejar uma arquitetura eficiente para atender aos requisitos da empresa.

6-Ter Conhecimento sobre o Domínio do Negócio

Imagine ser arquiteto em uma grande instituição financeira e não entender os termos financeiros comuns, como índice direcional médio, contratos aleatórios, aumento das taxas ou mesmo dívida não prioritária.

Sem esse conhecimento, um arquiteto não consegue se comunicar com os stakeholders e com os usuários do negócio, perdendo rápido a credibilidade.

6-Ter Conhecimento sobre o Domínio do Negócio

Os arquitetos mais bem-sucedidos que conhecemos são aqueles com conhecimento técnico amplo e prático, além de um bom conhecimento de certo domínio.

Tais arquitetos de software conseguem se comunicar bem com os executivos da diretoria e com os usuários comerciais, usando o conhecimento do domínio e uma linguagem que stakeholders conhecem e entendem.

6-Ter Conhecimento sobre o Domínio do Negócio

Isso acaba gerando uma grande confiança de que o arquiteto de software sabe o que está fazendo e tem competência para criar uma arquitetura eficiente e correta.

7-Ter Habilidades Interpessoais

Um arquiteto deve ter habilidades interpessoais excepcionais, inclusive trabalho em equipe, facilitação e liderança.

Ter habilidades excepcionais de liderança e interpessoais é uma expectativa difícil para a maioria dos desenvolvedores e dos arquitetos.

Como técnicos, os desenvolvedores e arquitetos gostam de resolver problemas técnicos, não problemas com pessoas.

7-Ter Habilidades Interpessoais

Contudo, segundo a famosa frase de Gerald Weinberg: “Não importa o que dizem, sempre é um problema com pessoas.”

Um arquiteto não deve apenas dar uma orientação técnica para a equipe, mas também liderar as equipes de desenvolvimento na implementação da arquitetura.

As habilidades de liderança são, pelo menos, metade do que é preciso para se tornar um arquiteto de software eficiente, não importa a função ou o cargo dele.

7-Ter Habilidades Interpessoais

O setor está repleto de arquitetos de software, todos competindo por um número limitado de posições na arquitetura.

Ter grandes habilidades de liderança e interpessoais é uma boa maneira de um arquiteto se diferenciar dos outros, de se destacar na multidão.

7-Ter Habilidades Interpessoais

Conhecemos muitos arquitetos de software que são excelentes técnicos, mas arquitetos ineficientes devido à incapacidade de liderar equipes, treinar e aconselhar desenvolvedores e de comunicar com eficiência ideias, decisões e princípios da arquitetura.

Nem é preciso dizer que tais arquitetos têm dificuldades de manter uma posição ou um trabalho.

8-Entender e Lidar Bem com Questões Políticas

Um arquiteto deve entender o clima político da empresa e conseguir lidar bem com ele. Pode parecer bem estranho falar sobre negociação e lidar com política corporativa em arquitetura de software.

Para mostrar como as habilidades de negociação são importantes e necessárias, considere o cenário em que um desenvolvedor toma a decisão de utilizar uma estratégia padrão para reduzir a complexidade ciclomática geral de certa parte de um código complexo.

8-Entender e Lidar Bem com Questões Políticas

Quem se importa com isso? Uma pessoa pode aplaudir o desenvolvedor por usar tal padrão, mas em quase todos os casos o desenvolvedor não precisa buscar aprovação para tal decisão.

Agora considere o cenário em que um arquiteto, responsável por um grande sistema de gerenciamento das relações com o cliente, tem problemas para controlar o acesso ao banco de dados a partir de outros sistemas, assegurar certos dados do cliente e fazer qualquer mudança no esquema do banco de dados porque muitos outros sistemas estão usando o banco de dados CRM.

8-Entender e Lidar Bem com Questões Políticas

Assim, o arquiteto toma a decisão de criar o que é chamado de silos de aplicação, em que cada banco de dados da aplicação é acessível apenas a partir da aplicação que possui esse banco de dados.

Tomar essa decisão dará ao arquiteto mais segurança e controle sobre os dados do cliente e sobre as mudanças. Porém, diferentemente do cenário do desenvolvedor anterior, essa decisão também será questionada por quase todos na empresa (com a possível exceção da equipe de aplicação CRM, claro).

8-Entender e Lidar Bem com Questões Políticas

Outras aplicações precisam dos dados de gerenciamento de clientes. Se essas aplicações não conseguem mais acessar diretamente o banco de dados, agora devem pedir ao sistema CRM para obter os dados, requerendo chamadas de acesso remotas via REST, SOAP ou algum outro protocolo de acesso remoto.

EXPECTATIVAS DE UM ARQUITETO

O ponto principal é que quase toda decisão que um arquiteto toma será questionada.

As decisões da arquitetura serão questionadas pelos proprietários do produto, pelos gerentes do projeto e pelos acionistas da empresa devido ao aumento de custos ou esforço (tempo) envolvido.

EXPECTATIVAS DE UM ARQUITETO

Essas decisões também serão questionadas por desenvolvedores que sentem que a abordagem deles é melhor.

Em qualquer caso, o arquiteto deve se envolver na política da empresa e aplicar habilidades básicas de negociação para ter a maioria das decisões aprovadas.

INTERSEÇÃO DA ARQUITETURA

INTERSEÇÃO DA ARQUITETURA

O escopo da arquitetura de software aumentou na última década para incluir cada vez mais responsabilidade e perspectiva.

Há uma década, a relação típica entre arquitetura e operações era contratual e formal, com muita burocracia.

INTERSEÇÃO DA ARQUITETURA

A maioria das empresas, tentando evitar a complexidade de hospedar suas próprias operações, muitas vezes as terceirizava, com obrigações contratuais para acordos de serviços, como tempo de atividade, escala, responsividade e muitas outras características importantes da arquitetura.

-PRÁTICAS DA ENGENHARIA

Por tradição, a arquitetura de software foi separada do processo de desenvolvimento usado para criar o software.

Dezenas de metodologias populares existem para criar um software (como Scrum, Programação Extrema, Lean), que não impactam muito a arquitetura de software.

-PRÁTICAS DA ENGENHARIA

Contudo, nos últimos anos, os avanços da engenharia têm impulsionado as preocupações do processo na arquitetura de software. É bom separar o processo de desenvolvimento de software das práticas de engenharia.

-PRÁTICAS DA ENGENHARIA

Com processo, referimo-nos a como as equipes são formadas e gerenciadas, como as reuniões são feitas e o fluxo de trabalho da organização; isso se refere à mecânica de como as pessoas se organizam e interagem.

-PRÁTICAS DA ENGENHARIA

Já as práticas de engenharia de software se referem às ações independentes do processo que têm um benefício ilustrado e repetido.

Por exemplo, a integração contínua é uma prática de engenharia comprovada que não conta com um processo em particular.

-PRÁTICAS DA ENGENHARIA

É importante focar nas práticas de engenharia. Primeiro, o desenvolvimento de software não tem muitos recursos das disciplinas de engenharia mais maduras.

Por exemplo, os engenheiros civis conseguem prever uma mudança estrutural com muito mais precisão do que os aspectos igualmente importantes da estrutura do software.

-PRÁTICAS DA ENGENHARIA

Segundo, um dos calcanhares de Aquiles do desenvolvimento de software é a estimativa, ou seja: quanto tempo, quantos recursos, quanto dinheiro?

-PRÁTICAS DA ENGENHARIA

Parte dessa dificuldade reside nas práticas de contabilidade antiquadas que não conseguem incluir a natureza exploratória desse desenvolvimento, mas outra parte é porque somos ruins com estimativas por tradição, pelo menos por causa das incógnitas desconhecidas.

-PRÁTICAS DA ENGENHARIA

As incógnitas desconhecidas são o inimigo dos sistemas de software. Muitos projetos começam com uma lista de incógnitas conhecidas: coisas que os desenvolvedores devem aprender sobre o domínio e a tecnologia que eles sabem que virão.

-PRÁTICAS DA ENGENHARIA

Porém os projetos também são vítimas das incógnitas desconhecidas: coisas que ninguém sabia que apareceriam, embora tenham aparecido inesperadamente.

Por isso os esforços de software do tipo “Grande Design no Início” sofrem: os arquitetos não conseguem se planejar para as incógnitas desconhecidas.

-PRÁTICAS DA ENGENHARIA

Todas as arquiteturas se tornam iterativas por causa das incógnitas desconhecidas. A metodologia ágil apenas reconhece isso e age mais cedo. Assim, embora o processo seja, em grande parte, separado da arquitetura, um processo iterativo fica melhor na natureza da arquitetura.

-PRÁTICAS DA ENGENHARIA

Em geral, o arquiteto também é o responsável técnico nos projetos, portanto determina as práticas de engenharia que a equipe usa.

-PRÁTICAS DA ENGENHARIA

ATENÇÃO:

- Auditoria
- Desempenho
- Segurança
- Requisitos
- Dados
- Legalidade
- Escalabilidade

-OPERAÇÕES / DEVOPS

A interseção recente e mais óbvia entre a arquitetura e os campos afins ocorreu com o advento do DevOps, orientado pelo repensar dos axiomas da arquitetura.

Por muitos anos, várias empresas consideraram as operações como uma função separada do desenvolvimento de software; muitas vezes, elas terceirizavam as operações como uma medida para economizar custos.

-OPERAÇÕES / DEVOPS

Definição de DevOps:

Um composto de Dev (desenvolvimento) e Ops (operações)

O DevOps é a união de pessoas, processos e tecnologias para fornecer continuamente valor aos clientes.

-OPERAÇÕES / DEVOPS

Muitas arquiteturas planejadas durante os anos 1990 e 2000 pressupunham que os arquitetos não podiam controlar as operações e foram criadas de modo defensivo em torno dessa restrição.

-OPERAÇÕES / DEVOPS

Contudo, há alguns anos, diversas empresas começaram a experimentar novas formas de arquitetura que combinam muitas questões operacionais com a arquitetura.

Por exemplo, nas arquiteturas mais antigas, como SOA baseado em uma arquitetura planejada para lidar com a escala elástica, complicando muito o processo.

-OPERAÇÕES / DEVOPS

Os criadores do estilo micro serviço da arquitetura perceberam que as operações lidam melhor com essas questões operacionais.

Criando uma ligação entre arquitetura e operações, os arquitetos conseguem simplificar o design.

-PROCESSO

Outro axioma é que a arquitetura de software é, em grande parte, ortogonal no processo de desenvolvimento de software; o modo como você cria o software (processo) tem pouco impacto na arquitetura dele (estrutura).

-PROCESSO

Dessa maneira, embora o processo de desenvolvimento de software que uma equipe usa tenha impacto na estrutura do software (em especial nas práticas da engenharia), historicamente ela tem sido considerada como separada na maior parte.

-PROCESSO

A maioria dos livros sobre arquitetura de software ignora o processo de desenvolvimento de software, fazendo suposições capciosas sobre coisas como previsibilidade.

-PROCESSO

Contudo, o processo pelo qual as pessoas desenvolvem o software tem impacto em muitas facetas da arquitetura de software.

Por exemplo, muitas pessoas nas últimas décadas adotaram metodologias de desenvolvimento ágil devido à natureza do software.

-PROCESSO

Os arquitetos de projetos ágeis podem realizar o desenvolvimento iterativo e, portanto, um ciclo de feedback mais rápido para as decisões. Isso, por sua vez, permite que os arquitetos sejam mais agressivos sobre a experimentação e outros conhecimentos que contam com o feedback.

-PROCESSO

Um aspecto crítico da arquitetura em que as metodologias ágeis se destacam é a reestruturação.

ARQUITETURA DE SOFTWARE

PARTE I: FUNDAMENTOS

- Pensamento Arquitetônico**
 - Modularidade**
- Definição das características da Arquitetura**
- Identificando as características da Arquitetura**
- Medindo e controlando as características da Arquitetura**
 - Escopo das características da Arquitetura**
 - Pensamento baseado em componentes**

-Pensamento Arquitetônico:

Um arquiteto pensa de modo diferente de um desenvolvedor, assim como um meteorologista vê as nuvens de modo diferente de um artista. Isso se chama pensamento arquitetônico. Infelizmente, muitos arquitetos acreditam que o pensamento arquitetônico se resume simplesmente a "pensar na arquitetura".

-Pensamento Arquitetônico:

Pensamento arquitetônico é muito mais do que isso. É ver as coisas com os olhos da arquitetura ou do ponto de vista arquitetural. Pensar como arquiteto envolve quatro aspectos principais.

Primeiro, é entender a diferença entre arquitetura e design, saber como colaborar com as equipes de desenvolvimento para a arquitetura funcionar.

-Pensamento Arquitetônico:

Segundo, é ter uma grande variedade de conhecimento técnico e ainda manter certo nível de profundidade técnica, permitindo que o arquiteto veja soluções e possibilidades que os outros não veem.

-Pensamento Arquitetônico:

Terceiro, é entender, analisar e reconciliar os trade-offs [significa estabelecer prioridades: identifique o que é mais importante] entre várias soluções e tecnologias.

Por fim, é entender a importância das motivações comerciais e como elas se traduzem em preocupações arquitetônicas.

-Pensamento Arquitetônico:

Amplitude Técnica: O escopo dos detalhes técnicos difere entre desenvolvedores e arquitetos. Diferentemente de um desenvolvedor, que deve ter bastante profundidade técnica para realizar seu trabalho, um arquiteto de software deve ter uma grande amplitude técnica para pensar como arquiteto e ver as coisas de um ponto de vista da arquitetura.

-Pensamento Arquitetônico:

Analizando os Trade-offs: Pensar como arquiteto é ver os trade-offs em toda solução, técnica ou outra, e analisá-las para determinar qual é a melhor solução.

Tudo na arquitetura é um trade-off, sendo por isso que a famosa resposta para toda pergunta de arquitetura no universo é “depende”.

-Modularidade:

Diferentes plataformas oferecem diferentes mecanismos de reutilização para o código, mas todas dão suporte a algum modo de agrupar o código afim em módulos.

Embora esse conceito seja universal na arquitetura de software, mostrou-se difícil de definir.

-Modularidade:

Entender a modularidade e suas muitas versões na plataforma de desenvolvimento escolhida é essencial para os arquitetos. Muitas ferramentas que temos para analisar a arquitetura (como métrica, funções de adequação e visualizações) contam com esses conceitos de modularidade.

Modularidade é um princípio organizacional.

-Modularidade:

O dicionário define módulo como “cada conjunto de partes padronizadas ou unidades independentes que podem ser usadas para construir uma estrutura mais complexa”.

Usamos a modularidade para descrever um agrupamento lógico de código.

-Modularidade:

Dada a importância da modularidade para arquitetos, eles precisam de ferramentas para entendê-la. São três os conceitos principais: coesão, acoplamento e conascência.

-Modularidade:

Coesão: se refere até que ponto as partes de um módulo devem estar contidas no mesmo módulo, ou seja, é uma medida da relação das partes entre si.

-Modularidade:

Acoplamento: O acoplamento aferente mede o número de conexões de entrada para um artefato do código (componente, classe, função etc.). O acoplamento eferente mede as conexões de saída.

-Modularidade:

Conascência: Ocorre quando um componente é modificado, e consequentemente, um segundo componente também 'sofre' uma modificação para manter a correção geral do sistema.

-Definição das Características da Arquitetura

O que diferencia a arquitetura de software da codificação e do design?
Muitas coisas, inclusive a função que os arquitetos têm ao definir as características da arquitetura, os aspectos importantes do sistema independentes do domínio do problema.

-Definição das Características da Arquitetura

Uma característica da arquitetura atende a três critérios:

Especifica uma consideração de design fora do domínio;

Influencia algum aspecto estrutural de design;

É essencial ou importante para o sucesso da aplicação.

-Identificando as Características da Arquitetura

Identificar as características que movem a arquitetura é um dos primeiros passos para criar uma arquitetura ou determinar a validade de uma existente.

-Identificando as Características da Arquitetura

Um arquiteto descobre as características da arquitetura pelo menos de três modos extraindo das preocupações, dos requisitos e do conhecimento implícito do domínio.

-Medindo e Controlando as Características da Arquitetura

Os arquitetos devem lidar com uma variedade enorme de características da arquitetura em todos os diferentes aspectos dos projetos de software.

-Medindo e Controlando as Características da Arquitetura

Medidas de processo
Medidas de pessoas [equipe]
Medidas de Tecnologia

-Escopo das Características da Arquitetura

Com o advento das técnicas de engenharia modernas e os estilos de arquitetura por elas permitidos, como micro serviços, o escopo das características da arquitetura são diferenciados.

-Escopo das Características da Arquitetura

Muitos desenvolvedores com familiaridade em ciência conhecem o conceito de quantum da física, a quantidade mínima de qualquer entidade física em uma interação.

-Pensamento Baseado em Componentes

Os componentes oferecem um mecanismo específico da linguagem para agrupar os artefatos, em geral aninhando-os para criar uma estratificação.

-Pensamento Baseado em Componentes

Os componentes também aparecem como subsistemas ou camadas na arquitetura, como a unidade de trabalho de implementação para muitos processadores de evento.

ARQUITETURA DE SOFTWARE

PARTE II: ESTILOS DE ARQUITETURA

-Fundamentos

-Estilo de Arquitetura em Camadas

-Estilo de Arquitetura Pipeline

-Estilo de Arquitetura Microkernel

-Estilo de Arquitetura Baseada em Serviços

-Estilo de Arquitetura Orientada a Eventos

-Estilo de Arquitetura Baseada em Espaço

-Arquitetura Orientada a Serviços e Baseada em Orquestração

-Arquitetura de Micros Serviços

-Escolhendo o Estilo Certo

Fundamentos

Os estilos de arquitetura, por vezes chamados de padrões de arquitetura, descrevem uma relação nomeada de componentes que cobrem várias características da arquitetura.

Um nome de estilo de arquitetura, parecido com os padrões de projeto, cria um único nome que atua como atalho entre os arquitetos experientes.

Fundamentos

Por exemplo, quando um arquiteto fala sobre um bloco monolítico em camadas, seu objetivo na conversa é entender os aspectos da estrutura, quais características da arquitetura funcionam bem (e quais podem causar problemas), os modelos comuns de implementação, as estratégias de dados e muitas outras informações.

Fundamentos

Assim, os arquitetos devem estar familiarizados com os nomes básicos dos estilos de arquitetura fundamentais e genéricos. Cada nome captura muitos detalhes entendidos, uma das finalidades dos padrões de projeto.

Fundamentos

#Arquitetura Unitária:

#Cliente/Servidor:

- desktop + servidor de dados
- navegador + servidor web
- três camadas
- e hoje? = arquiteturas distribuídas

Fundamentos

#Arquiteturas Monolíticas x Distribuídas [mitos e falácias]

a rede é confiável...

a latência é zero...

a largura de banda é infinita...

a rede é segura...

a topologia nunca muda...

existe apenas um administrador...

o custo do transporte é zero [não é latência e sim acessos]...

a rede é homogênea...

Estilo de Arquitetura em Camadas

#Topologia

Apresentação: A camada de apresentação é responsável pela tradução, compressão e criptografia dos dados para garantir a compatibilidade entre diferentes sistemas.

É responsável pela apresentação, interação do usuário e recebimento de dados processados pelo software bem como suas funções.

Estilo de Arquitetura em Camadas

#Topologia

Controle: A camada de controle é responsável por comandar o fluxo da apresentação servindo como uma camada intermediária entre a camada de apresentação e a lógica.

Estilo de Arquitetura em Camadas

#Topologia

Persistência: A camada de persistência é a responsável pela lógica de negócios, pois se refere a implementação de regras do negócio ou requisitos do sistema.

Estilo de Arquitetura em Camadas

#Topologia

Banco de Dados: A camada de dados inclui as técnicas de persistência de dados (compartilhamento de arquivos, servidores de banco de dados e demais).

É a camada de acesso a dados que encapsula os mecanismos e expõe os dados.

Estilo de Arquitetura em Camadas

#Camadas de isolamento [aberta ou fechada]

aberta = independente

fechada = deve respeitar o precedente

Estilo de Arquitetura Pipeline

Os canais e os filtros coordenam-se de um modo específico, com os canais formando uma comunicação unidirecional entre os filtros, em geral de ponto a ponto.

Estilo de Arquitetura Pipeline

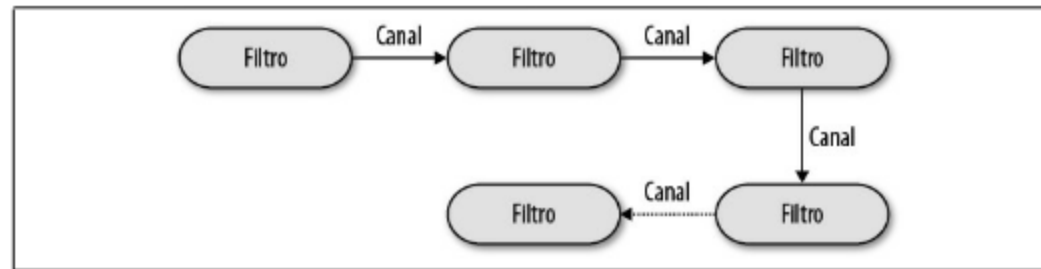


Figura 1 1-1. Topologia básica da arquitetura pipeline

Estilo de Arquitetura Microkernel

O estilo de arquitetura microkernel é uma estrutura monolítica relativamente simples consistindo em dois componentes da arquitetura: um sistema central e componentes de plug-in.

Estilo de Arquitetura Microkernel

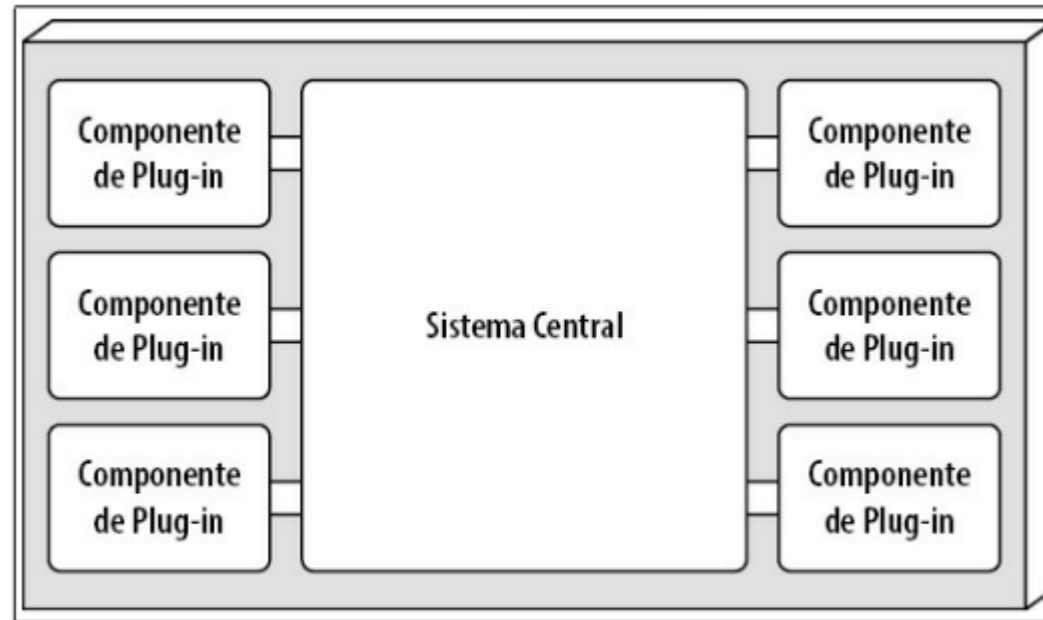


Figura 12-1. Componentes básicos do estilo da arquitetura microkernel

Estilo de Arquitetura Baseada em Serviços

A arquitetura baseada em serviços é uma combinação do estilo microserviços e é considerada um dos estilos mais pragmáticos, em grande parte devido à sua flexibilidade arquitetural.

Estilo de Arquitetura Baseada em Serviços

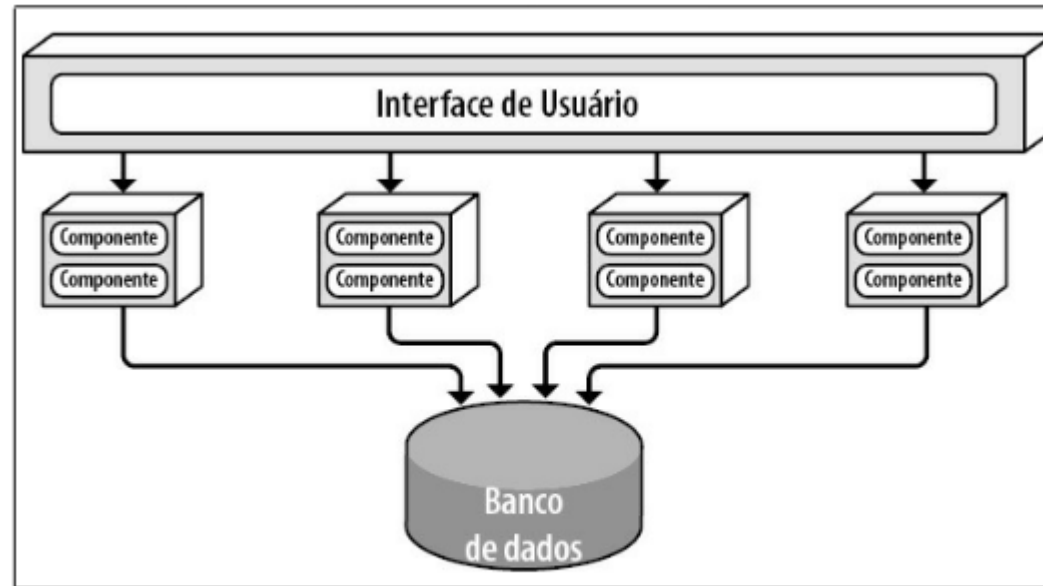


Figura 13-1. Topologia básica do estilo de arquitetura baseada em serviços

Estilo de Arquitetura Orientada a Eventos

O estilo de arquitetura orientada a eventos é uma arquitetura assíncrona distribuída e popular usada para produzir aplicativos de alto desempenho e escala.

Estilo de Arquitetura Orientada a Eventos

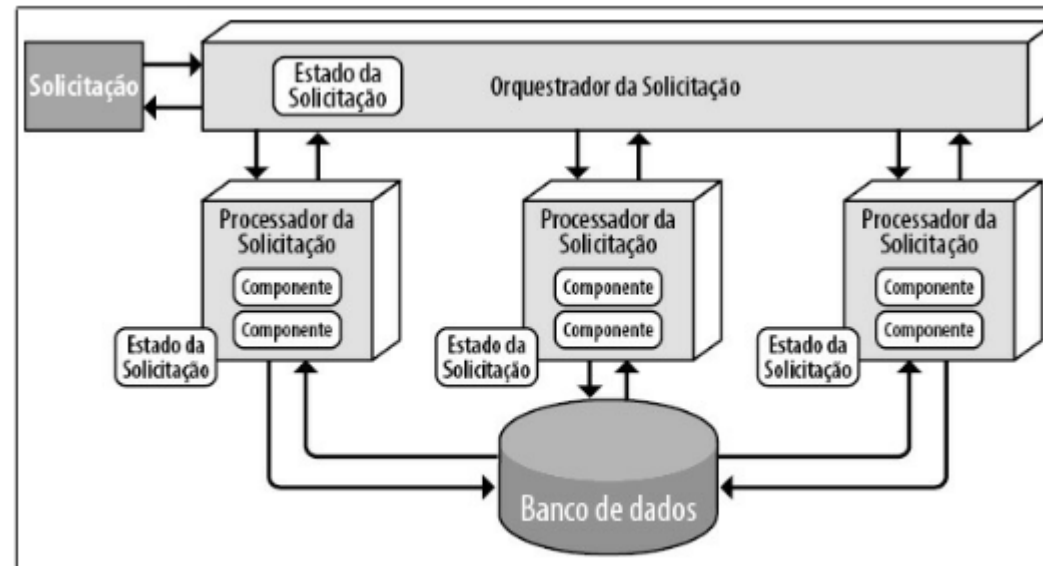


Figura 14-1. Modelo baseado em requisição

Estilo de Arquitetura Baseada em Espaço

A maioria dos aplicativos para web segue o mesmo fluxo geral: uma requisição do navegador chega ao servidor web, então no servidor do aplicativo e, por fim, no servidor do banco de dados.

Estilo de Arquitetura Baseada em Espaço

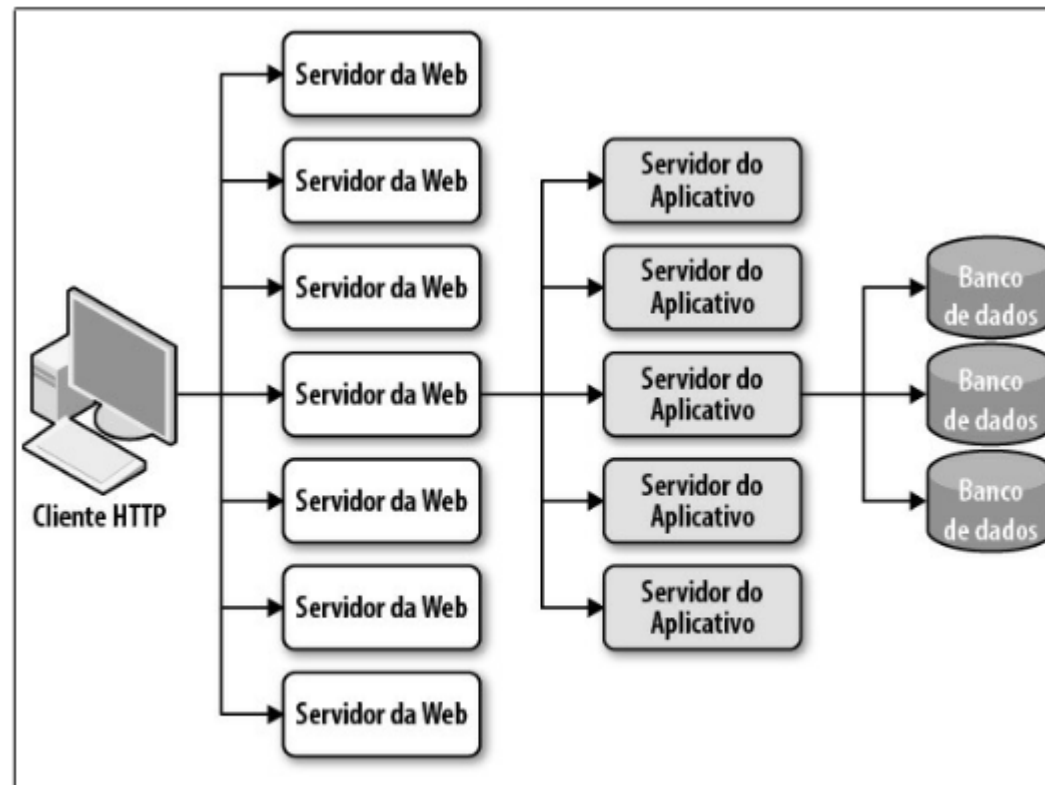


Figura 15-1. Limites da escalabilidade dentro de uma topologia tradicional para a web

-Arquitetura Orientada a Serviços e Baseada em Orquestração

A combinação das forças externas que costumam influenciar as decisões da arquitetura, junto a uma filosofia organizacional lógica, mas definitivamente desastrosa, condenou essa arquitetura à irrelevância.

Mas é um ótimo exemplo de como certa ideia organizacional pode ter um sentido lógico, embora impeça as partes mais importantes do processo de desenvolvimento.

-Arquitetura Orientada a Serviços e Baseada em Orquestração

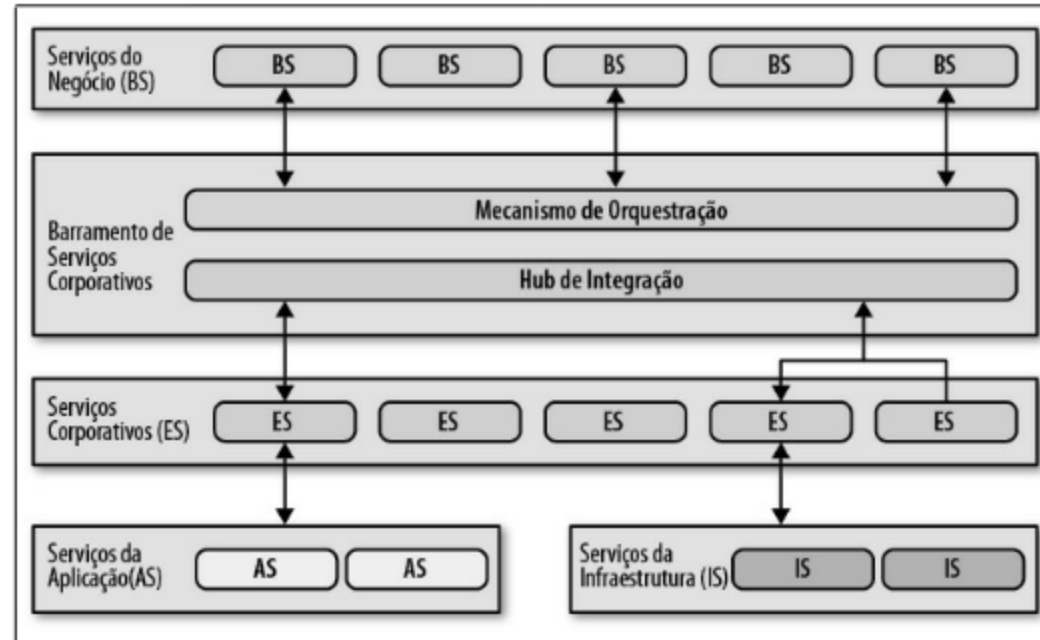


Figura 16-1. Topologia da arquitetura orientada a serviços e baseada em orquestração

Arquitetura de Micros Serviços

A arquitetura de micros serviços é muito inspirados pelas ideias no DDD (domain-driven design), um processo de design lógico para projetos de software.

Um conceito em particular do DDD, o contexto delimitado, certamente inspirou os micros serviços.

Arquitetura de Micros Serviços

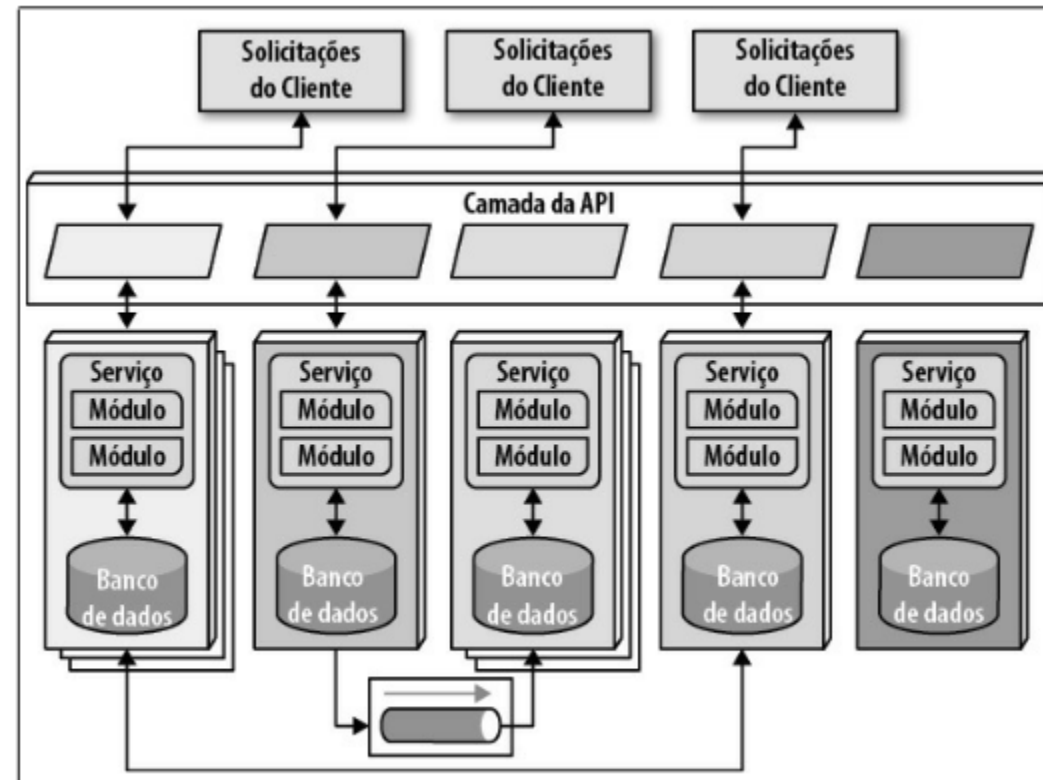


Figura 17-1. Topologia do estilo da arquitetura de microsserviços

Escolhendo o Estilo Certo

Nada é mais contextual segundo vários fatores em uma organização e qual software ela cria.

Escolher um estilo de arquitetura representa o ápice da análise e do pensamento estratégico para as características da arquitetura, considerações do domínio, metas e muitas outras coisas.

ARQUITETURA DE SOFTWARE

PARTE III: TÉCNICAS E HABILIDADES SOCIAIS

- Decisões da Arquitetura**
- Analisando o Risco da Arquitetura**
- Diagramando e Apresentando a Arquitetura**
- Tornando as Equipes Eficientes**
- Habilidades de Negociação e Liderança**
- Desenvolvendo uma Trajetória Profissional**

-Decisões da Arquitetura

Uma das expectativas essenciais de um arquiteto é tomar decisões de arquitetura.

Em geral, essas decisões envolvem a estrutura da aplicação ou do sistema, mas podem envolver decisões de tecnologia também, em particular quando tais decisões impactam as características da arquitetura.

-Analisando o Risco da Arquitetura

Toda arquitetura tem um risco associado, seja envolvendo disponibilidade, escalabilidade ou integridade de dados.

Analisar esse risco é uma das principais atividades da arquitetura.

Analizando-o continuamente, o arquiteto pode lidar com as deficiências na arquitetura e tomar uma ação corretiva para mitigar o risco.

-Diagramando e Apresentando a Arquitetura

Os arquitetos recém-formados normalmente comentam sobre o quão surpresos eles estão com a variedade do trabalho, que vai além do conhecimento técnico e da experiência, o que lhes permitiu exercerem a função de arquiteto em primeiro lugar.

-Diagramando e Apresentando a Arquitetura

A comunicação efetiva se torna essencial para o sucesso de um arquiteto.

Independentemente do quanto brilhantes sejam as ideias técnicas de um arquiteto, se ele não consegue convencer os gerentes a financiá-las e os desenvolvedores a criá-las, esse brilhantismo nunca se manifestará.

-Diagramando e Apresentando a Arquitetura

Diagramar e apresentar as arquiteturas são duas habilidades sociais essenciais para os arquitetos.

Embora existam livros inteiros sobre cada tópico, veremos três pontos de destaque:

1-Tornando as Equipes Eficientes

Além de criar uma arquitetura técnica e tomar decisões de arquitetura, um arquiteto de software também é responsável por orientar a equipe de desenvolvimento na implementação da arquitetura.

2-Habilidades de Negociação e Liderança

É difícil ter habilidades de negociação e liderança.

Leva anos de experiência em liderança, prática e “lições aprendidas” para obter as habilidades necessárias para se tornar um arquiteto de software eficiente.

3-Desenvolvendo uma Trajetória Profissional

Tornar-se “arquiteto de software” requer tempo, esforço e comprometimento. Contudo, para gerenciar a trajetória profissional depois de se tornar “arquiteto”, é igualmente exigido cuidados diários.

Um arquiteto deve continuar a aprender durante sua carreira, pois o mundo da tecnologia muda em um ritmo vertiginoso.

ARQUITETURA DE SOFTWARE

FIM